

Document Intelligence Platform

Architecture & Design Manual

A comprehensive guide to the architecture, design decisions, and technology choices behind an enterprise AI-powered document reasoning platform.

Version 1.0 — June 2026

Author: Cognitera Engineering

Stack: Spring Boot 3.3 • Ollama • Qdrant • Neo4j • PostgreSQL

Table of Contents

1. Executive Summary

2. What the Platform Does

2.1 Core Capabilities

2.2 User Journey

2.3 Why This Matters

3. High-Level Architecture

3.1 Architectural Diagram

3.2 Multi-Module Maven Structure

3.3 Module Responsibilities

3.4 Layered Architecture Pattern

4. Technology Stack & Rationale

4.1 Why Spring Boot 3.3

4.2 Why Ollama for LLM Inference

4.3 Why Thymeleaf Over React

4.4 Why PostgreSQL (with pgvector)

4.5 Why Neo4j for the Knowledge Graph

4.6 Why Qdrant for Vector Search

4.7 Why Java 21

5. Search Architecture

5.1 Hybrid Search: Concept

5.2 Keyword Search Path

5.3 Semantic (Vector) Search Path

5.4 Fusion & Reranking Algorithm

5.5 Chunking Strategy

5.6 Query Intent Classification

6. AI & RAG Pipeline

6.1 Retrieval-Augmented Generation Flow

6.2 Prompt Engineering Architecture

6.3 Grounding & Attribution

6.4 Temporal Consistency & Claim Validation

6.5 Confidence Scoring

7. Knowledge Graph (Neo4j)

7.1 Role of the Knowledge Graph

7.2 Graph Data Model

7.3 Integration Pattern

8. Data & Persistence Layer

8.1 PostgreSQL Schema Strategy

8.2 Why We Need a Relational Database

8.3 Document Storage & Versioning

9. Security Architecture

9.1 JWT-Based Authentication

9.2 Authorization Model

9.3 Audit Trail System

10. Internationalization (i18n)

11. Ingestion Pipeline

12. Workspace System

13. Testing Strategy

14. Deployment & Operations

15. Why This Architecture Excels

16. Key Design Decisions Summary

1. Executive Summary

A modern enterprise platform that ingests, indexes, analyzes, and reasons over document collections using a hybrid of classical search, vector similarity, knowledge graphs, and large language models — all running on privately hosted, open-source infrastructure.

The **Document Intelligence Platform** is a Spring Boot 3.3 application that transforms how organizations interact with their document repositories. It combines **hybrid search** (keyword + semantic vector), a **Neo4j knowledge graph**, and **retrieval-augmented generation (RAG)** via locally hosted Ollama LLMs to let users ask natural-language questions and receive evidence-backed, citation-grounded answers drawn from their own documents.

Key insight: This platform is not a chatbot bolted onto a search engine. It is a purpose-built document reasoning system where every AI-generated answer is traceable to specific source passages, validated for temporal consistency, and scored for confidence.

The platform serves three primary audiences:

1. **Knowledge workers** — who need to find and reason over documents across workspaces.
2. **Legal/compliance professionals** — who require citation-grounded answers with provenance chains.
3. **AI consultants & architects** — who want a reference implementation showcasing modern AI engineering patterns.

2. What the Platform Does

2.1 Core Capabilities

Capability	Description
Document Ingestion	Upload documents (PDF, DOCX, plain text) with automatic text extraction, chunking, and indexing into both keyword and vector stores.
Hybrid Search	Query across all documents using keyword search, semantic vector search, or a fused hybrid mode. Results are reranked and scored.
AI-Powered Q&A	Ask natural-language questions. The platform retrieves relevant document passages, augments the LLM prompt with that context, and produces citation-backed answers.
Knowledge Graph	A Neo4j graph database captures relationships between structured knowledge entries (e.g., legal statutes, regulatory references), enabling graph traversal queries.
Workspace Management	Organize documents into workspaces with multi-phase workflows: Setup → Upload → Ingest → Analysis → Review → Complete.
Document Versioning	Full version history for every document, with the ability to add new versions while preserving all previous ones.
Audit Trail	Every significant action (login, upload, search, AI inference, ingestion) is recorded with correlation IDs for end-to-end traceability.
Internationalization	UI available in English, German, and French with session-based locale switching.
Semantic Code Indexing	A companion module indexes Java source code for semantic code search and AI-assisted code understanding.

2.2 User Journey

A typical user session flows through these steps:

1. **Authenticate** — Login with JWT-based credentials (access + refresh token).
2. **Create a Workspace** — Define a workspace with a name, description, and type (e.g., Legal Review, Research).
3. **Upload Documents** — Drag-and-drop PDFs or DOCX files. The platform extracts text, chunks it, generates embeddings, and indexes into PostgreSQL + Qdrant.
4. **Search** — Use keyword, semantic, or hybrid search to find relevant passages across all documents.

5. **Ask AI Questions** — Pose natural-language questions. The RAG pipeline retrieves relevant chunks, augments the prompt, queries the LLM, and grounds the answer with citations.
6. **Review & Export** — Browse AI answers with their source citations. Review workspace analysis results.
7. **Audit** — View the complete audit trail of all actions for compliance and traceability.

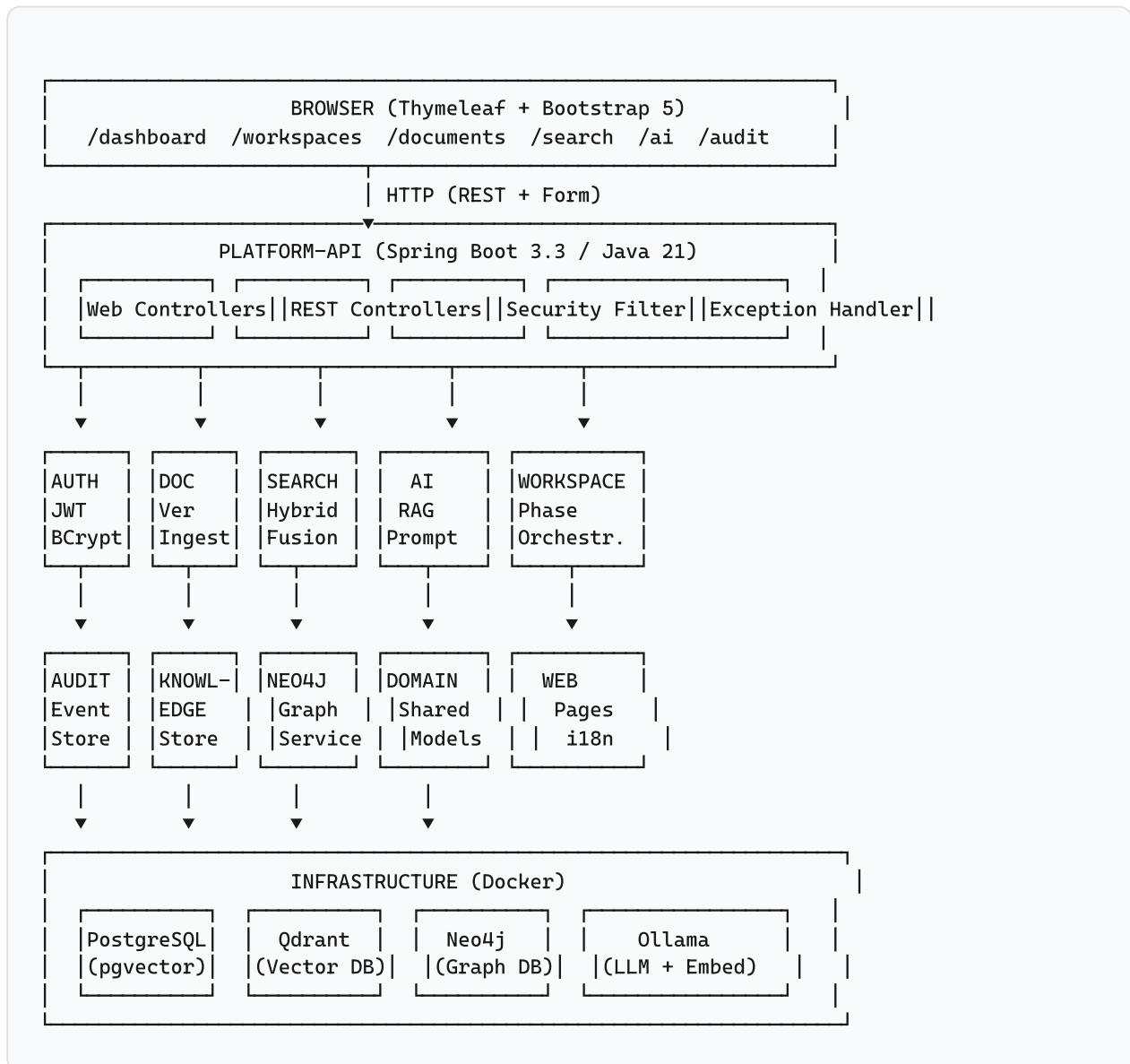
2.3 Why This Matters

Organizations sit on millions of documents — contracts, regulatory filings, legal opinions, research papers, internal policies. Traditional keyword search finds documents but not answers. RAG-based AI can synthesize answers, but without proper grounding, it hallucinates. This platform bridges the gap:

evidence-backed AI reasoning over your own document corpus, running on infrastructure you control.

3. High-Level Architecture

3.1 Architectural Diagram



3.2 Multi-Module Maven Structure

The platform is organized as a **12-module Maven multi-module project**. Each module has a clear, bounded responsibility with its own `api/` (interfaces), `application/` (implementations), and `infrastructure/` (persistence) packages. This enforces the **dependency inversion principle** — modules depend on interfaces, not implementations.

Module	Role	Key Dependencies
<code>platform-domain</code>	Shared domain models, value objects	None (root dependency)
<code>platform-audit</code>	Audit event definitions, repository, publisher API	domain
<code>platform-auth</code>	JWT authentication, user accounts, session tokens	domain, audit
<code>platform-document</code>	Document CRUD, versioning, ingestion pipeline	domain, audit
<code>platform-search</code>	Hybrid search, chunking, embeddings, Qdrant integration	domain, document, audit
<code>platform-ai</code>	RAG pipeline, prompting, grounding, Ollama chat	domain, search, audit
<code>platform-workspace</code>	Workspace CRUD, phase orchestration, timeline extraction	domain, document
<code>platform-knowledge</code>	Knowledge entry storage, ingestion from external sources	domain
<code>platform-neo4j</code>	Neo4j graph service: nodes, relationships, traversal	knowledge
<code>platform-web</code>	Thymeleaf page controllers, i18n messages, CSS	All above modules
<code>platform-api</code>	REST controllers, DTOs, Spring Boot entry point	All above modules
<code>semantic-indexing</code>	Standalone code indexing app (separate Spring Boot app)	Independent

3.3 Module Responsibilities in Detail

`platform-ai` — The AI Engine

This is the intellectual core of the platform. It implements a **full RAG pipeline** with 20+ service interfaces:

- `QueryIntentClassifier` — Routes queries by intent (factual, analytical, procedural, index inspection).
- `RetrievalAugmentationService` — Fetches relevant sources, authority references, hierarchy, and timeline.
- `ContextAssembler` — Assembles the full prompt context from retrieval results.
- `PromptBuilder` — Constructs the final prompt with retrieved context and instructions.
- `ChatCompletionProvider` — Interface for LLM inference (Ollama implementation).
- `GroundingService` — Reattributes sources, computes confidence, produces reasoned answers.
- `TemporalConsistencyValidator` — Checks that AI claims don't contradict temporal constraints.
- `ClaimValidator` — Validates that claims are supported by retrieved authority references.
- `StructuredAnswerAssembler` — Formats final answers with citations, confidence, and metadata.

platform-search — The Retrieval Engine

The search module provides a multi-modal retrieval architecture:

- `HybridRetrievalService` — Combines keyword + vector results with fusion scoring.
- `SentenceAwareChunkingStrategy` — Splits documents at sentence boundaries with configurable overlap.
- `OllamaEmbeddingProvider` — Generates embeddings via Ollama's `/api/embeddings`.
- `QdrantVectorSearchProvider` — Indexes and searches vectors in Qdrant.
- `JpaKeywordSearchProvider` — Full-text keyword search via PostgreSQL.
- `OllamaRerankingProvider` — Cross-encoder reranking via Ollama.

3.4 Layered Architecture Pattern

Every module follows the same internal layering:

```
api/          - Interfaces (contracts)
application/- Implementations (services)
infrastructure/ - Persistence, external I/O
model/        - Domain objects, value types
```

This is inspired by **Ports & Adapters (Hexagonal Architecture)**. The `api/` package defines the ports; `application/` contains the domain logic; `infrastructure/` contains the adapters (JPA repositories, Ollama HTTP clients, Qdrant REST clients).

4. Technology Stack & Rationale

4.1 Why Spring Boot 3.3

Spring Boot was chosen as the application framework for several deliberate reasons:

Factor	Rationale
Enterprise Maturity	Spring Boot has 15+ years of production hardening. Its dependency injection, transaction management, security, and configuration systems are battle-tested in Fortune 500 deployments.
Ecosystem Breadth	Spring Data JPA, Spring Security, Spring AI, and Spring Web provide integrated, well-documented solutions for every layer of the application — no stitching together disparate libraries.
Type Safety	Java's static typing catches entire categories of bugs at compile time that dynamic languages (Python, JavaScript) defer to runtime. In a system processing legal documents, this matters.
Virtual Threads (Java 21)	Spring Boot 3.3 on Java 21 supports Project Loom virtual threads, enabling high-concurrency I/O operations (embedding generation, LLM calls, vector search) without reactive programming complexity.
Modularity	Maven multi-module builds enforce compile-time boundaries between subsystems. You cannot accidentally create a circular dependency.
Operations	Spring Boot Actuator provides health checks, metrics, and monitoring endpoints that integrate directly with Kubernetes, Prometheus, and Grafana.

Contrast: Python (FastAPI/Flask) would have been faster to prototype but would lack the compile-time safety, transaction management, and enterprise integration capabilities that Spring provides out of the box. For a platform that must be *correct* (not just fast to build), Java + Spring is the right engineering choice.

4.2 Why Ollama for LLM Inference

Ollama is used as the **sole LLM provider** for both chat completions and embedding generation. This is a deliberate architectural decision with significant implications:

Factor	Rationale
--------	-----------

Data Sovereignty	All inference runs locally. Documents never leave the organization's infrastructure. This is non-negotiable for legal, medical, and government use cases.
Zero API Costs	No OpenAI/Anthropic API bills. The marginal cost of a query is electricity. For organizations processing thousands of documents daily, this saves millions.
Model Flexibility	Swap models without code changes — change <code>OLLAMA_CHAT_MODEL</code> from <code>qwen2.5:14b</code> to <code>llama3.1:70b</code> or any other Ollama-compatible model. No vendor lock-in.
Provider Abstraction	The <code>ChatCompletionProvider</code> interface abstracts the LLM backend. The <code>OllamaChatCompletionProvider</code> is one implementation; switching to an OpenAI-compatible API requires only a new adapter class.
Offline Capability	The platform works in air-gapped environments. No internet connection required for inference.

Technical Integration

The Ollama integration is straightforward HTTP-based communication:

```
// Embedding generation
POST http://localhost:11434/api/embeddings
{"model": "nomic-embed-text", "prompt": "chunk text here"}

// Chat completion
POST http://localhost:11434/api/chat
{"model": "qwen2.5:14b", "messages": [{"role": "user", "content": "..."}], "stream": false}
```

The `OllamaChatCompletionProvider` uses Java's built-in `java.net.http.HttpClient` (no additional dependencies) to communicate with Ollama's REST API. The `OllamaEmbeddingProvider` uses Spring's `RestClient` for the same purpose.

4.3 Why Thymeleaf Over React

This is one of the most important architectural decisions — and one that often surprises people expecting a modern SPA. The choice is deliberate:

Factor	Thymeleaf (chosen)	React (rejected)
Complexity Budget	Server-side rendering with no JavaScript build step. One process to deploy.	Requires a separate frontend build (webpack/vite), Node.js runtime, API versioning, CORS handling.
Security Model	Session-based form login with CSRF tokens handled by Spring Security. Server controls	JWT stored in browser localStorage — vulnerable to XSS. Requires careful token

	all rendering.	management.
SEO & Accessibility	Fully rendered HTML on first load. Works without JavaScript.	Requires SSR framework (Next.js) for equivalent behavior — adding another build pipeline.
i18n Integration	Spring's MessageSource + Thymeleaf's <code># {key}</code> syntax. No extra tooling.	Requires i18next or similar library, separate translation loading, locale detection logic.
Team Profile	Java developers can maintain the full stack. No frontend specialists required.	Requires JavaScript/React expertise on the team.
Deployment	Single JAR file. <code>java -jar platform-api.jar</code> .	Separate static hosting or embedded Node server. More moving parts.

Design philosophy: Choose the simplest tool that meets the requirements. This platform's value is in its AI and search capabilities, not its UI framework. Thymeleaf delivers a professional, internationalized, accessible UI with zero JavaScript build tooling. For an enterprise document platform, this is the engineering-correct choice. React would be appropriate if the UI required rich real-time interactivity (collaborative editing, websocket-driven updates, complex client-side state) — but this platform's UI is primarily form-driven and page-based.

4.4 Why PostgreSQL (with pgvector)

PostgreSQL serves as the **primary relational database** for all structured data: users, documents, workspaces, chunks, audit events, knowledge entries, ingestion jobs, and timelines. We use the **pgvector** extension (pgvector/pgvector:pg16 Docker image) to enable vector storage alongside relational data.

Factor	Rationale
ACID Transactions	Document ingestion spans multiple tables (documents, versions, chunks, metadata). PostgreSQL guarantees atomicity — a failed ingestion leaves no partial state.
Rich Querying	Spring Data JPA Specifications enable dynamic, type-safe query building. Audit queries filter by actor, tenant, event type, and date range — all composable.
pgvector	Enables vector similarity operations directly in SQL. While we primarily use Qdrant for vector search, pgvector provides a fallback and enables hybrid queries that join vector similarity with relational filters.
Operational Simplicity	One database technology for 80% of storage needs. PGAdmin provides a management UI. Backups, replication, and monitoring are well-understood.

4.5 Why Neo4j for the Knowledge Graph

Neo4j serves a **specialized role** distinct from PostgreSQL. It stores structured knowledge entries (e.g., legal statutes, regulatory frameworks, authority hierarchies) as a graph, enabling:

- **Hierarchy traversal:** Navigate from a broad statute down to its subsections and related clauses via `PART_OF_HIERARCHY` relationships.
- **Concept-based discovery:** Traverse `RELATED_CONCEPT` edges to find knowledge entries connected by shared legal or regulatory concepts.
- **Graph-native queries:** Cypher queries like "find all entries up to 3 hops from Entry X via `RELATED_CONCEPT` relationships" express graph traversals that would be painful in SQL.

Why not just use PostgreSQL for this? Relational databases model hierarchies via recursive CTEs or adjacency lists — they work, but query performance degrades with depth and the queries become hard to read. Neo4j's native graph storage and Cypher query language make "*find all related concepts within N hops*" a one-line query. The knowledge graph is inherently graph-shaped data; using a graph database is the right tool for the job.

4.6 Why Qdrant for Vector Search

Qdrant is a **purpose-built vector database** written in Rust, chosen over alternatives (Pinecone, Weaviate, Milvus, Chroma) for specific reasons:

Factor	Rationale
Self-Hosted	Runs as a Docker container alongside the app. No external API dependency. Matches the data sovereignty requirement.
Performance	Rust implementation provides low-latency similarity search even with millions of vectors. HNSW index for approximate nearest neighbor (ANN) search.
Filtering	Payload-based filtering allows narrowing vector search by document metadata (tenant, document type, date range) without post-processing.
REST API	Clean REST API — no gRPC requirement. Integrates with Spring's RestClient using standard JSON serialization.
No Heavy Dependencies	Single binary in a Docker container. No ZooKeeper, no etcd, no external coordination service.

4.7 Why Java 21

Java 21 is the current **Long-Term Support (LTS)** release. Key features leveraged:

- **Virtual Threads (Project Loom):** Available for high-concurrency I/O without reactive programming.

- **Record Classes:** Used extensively for DTOs and value objects (see `AiResponse` , `SearchQuery` , `ReasonedAnswer`).
- **Pattern Matching:** Cleaner `instanceof` expressions and switch expressions.
- **Sealed Classes:** Enable exhaustive type checking for domain models like `QueryIntent` and `SearchMode` .

5. Search Architecture

5.1 Hybrid Search: Concept

The platform implements **hybrid search** — the fusion of keyword-based (lexical) search and semantic (vector) search. Neither approach alone is sufficient:

Keyword Search (BM25/LIKE)

- ✓ Exact term matching
- ✓ Fast for known terminology
- ✓ Predictable results
- X Misses synonyms & paraphrases
- X No semantic understanding
- X "Car" won't match "automobile"

Semantic Search (Vector)

- ✓ Understands meaning & intent
- ✓ Finds conceptually related content
- ✓ Multilingual capability
- X Can miss exact matches
- X Computationally expensive
- X Embedding model quality matters

Hybrid search combines both: run keyword and vector search in parallel, merge results with a weighted fusion algorithm, and rerank the combined list. This gives users the precision of keyword search plus the recall of semantic search.

5.2 Keyword Search Path

```
User Query → QueryIntentClassifier → JpaKeywordSearchProvider
      ↓
PostgreSQL LIKE/ILIKE on DocumentChunkEntity.text
(with optional LegalQueryExpander for domain-specific synonyms)
      ↓
List<RetrievalCandidate> with keywordScore
```

5.3 Semantic (Vector) Search Path

```
User Query → OllamaEmbeddingProvider.embed(query) → float[768]
      ↓
QdrantVectorSearchProvider.search(queryVector)
POST /collections/{name}/points/search {vector: [...], limit: N, with_payload: true}
      ↓
Qdrant returns top-K points by cosine similarity + payload
      ↓
List<RetrievalCandidate> with vectorScore
```

5.4 Fusion & Reranking Algorithm

The `DefaultHybridRetrievalService.merge()` method implements a **weighted reciprocal rank fusion** with document-type and intent-aware boosting:

```
rankingScore = (keywordScore * 0.40) + (vectorScore * 0.40)
               + (confidenceScore * 0.20)
               * docTypeWeight
               * authorityBoost
```

The fusion pipeline has four stages:

1. **Keyword retrieval** — PostgreSQL full-text / LIKE search
2. **Vector retrieval** — Qdrant ANN with embedding similarity
3. **Merge & deduplicate** — Combine by chunk ID, keeping the best score per source
4. **Rerank (optional)** — Cross-encoder reranker via Ollama for relevance refinement

5.5 Chunking Strategy

The `SentenceAwareChunkingStrategy` splits documents into semantically coherent chunks:

- **Sentence-boundary aware:** Chunks break at sentence endings, not mid-sentence.
- **Configurable chunk size:** Default target size with overlap between consecutive chunks.
- **Overlap:** Each chunk shares N characters with its neighbors to prevent information loss at boundaries.
- **Metadata preservation:** Each chunk carries its document ID, version, position index, document type, and source metadata.

Chunk metadata is stored in both PostgreSQL (`DocumentChunkEntity`) and as Qdrant payload for filtering during vector search.

5.6 Query Intent Classification

Before executing a search, the `QueryIntentClassifier` analyzes the user's query and classifies it into one of several intents:

Intent	Example	Routing
FACTUAL	"What is the penalty for breach of contract?"	RAG pipeline with authority grounding
ANALYTICAL	"Compare the liability provisions across these three contracts."	RAG with multi-source context assembly

PROCEDURAL	"How do I file a motion for summary judgment?"	RAG with timeline extraction
INDEX_INSPECTION	"What documents are in the index?"	Routed to IndexInspectionService (bypasses RAG)
CORPUS_DISCOVERY	"Show me all documents about employment law."	Routed to IndexInspectionService (bypasses RAG)

This intent routing prevents wasting LLM inference on queries that are better answered by direct index inspection, saving both latency and compute.

6. AI & RAG Pipeline

6.1 Retrieval-Augmented Generation Flow

The AI pipeline is the platform's most sophisticated subsystem. Each user question flows through an 8-stage pipeline:

1. INTENT CLASSIFICATION
`QueryIntentClassifier.classify(question) → QueryIntent`
 (Route INDEX_INSPECTION/CORPUS_DISCOVERY queries away from RAG)
2. RETRIEVAL
`RetrievalAugmentationService.retrieve(request)`
 - └─ `SourceOrchestrationService` → `HybridRetrievalService`
 - └─ `AuthorityRetrievalService` → Knowledge Graph lookup
 - └─ `FindingHierarchyService` → Hierarchy extraction
 - └─ `CommunicationTimelineBuilder` → Timeline construction
3. CONTEXT ASSEMBLY
`ContextAssembler.assemble(request, retrievalContext)`
 → PromptContext with sources, authorities, hierarchy, timeline
4. PROMPT CONSTRUCTION
`PromptBuilder.build(promptContext)`
 → Full prompt with instructions, context, and guardrails
5. LLM INFERENCE
`ChatCompletionProvider.complete(prompt, capabilities)`
 → `OllamaChatCompletionProvider` → Ollama /api/chat
6. POST-INFERENCE VALIDATION
 - └─ `TemporalConsistencyValidator.validate()`
 - └─ `ClaimValidator.validate()`
 → Warnings appended if inconsistencies detected
7. GROUNDING
`GroundingService.ground(rawAnswer, retrievalContext)`
 → ReasonedAnswer with source citations and confidence scores
8. RESPONSE ASSEMBLY
 → AiResponse with ReasonedAnswer + InferenceMetadata

6.2 Prompt Engineering Architecture

The prompt system is designed for **reproducibility and auditability**:

- **Templated prompts:** The `PromptBuilder` interface abstracts prompt construction. Each implementation has a `templateVersion()` that is recorded in audit logs — every AI answer can be traced to the exact prompt template version that produced it.
- **Context injection:** Retrieved source passages, authority references, hierarchy, and timeline are injected into the prompt in a structured format with clear delimiters.
- **Grounding instructions:** The prompt instructs the LLM to cite specific sources and distinguish between statements supported by evidence vs. general knowledge.
- **Guardrails:** Instructions to refuse to speculate, acknowledge uncertainty, and flag temporal inconsistencies.

6.3 Grounding & Attribution

Grounding is the process of linking claims in the AI's answer back to specific source passages. The `GroundingService` :

1. Parses the raw LLM output for implicit and explicit citations.
2. Matches claimed sources against the `RetrievalContext` (the passages that were actually provided in the prompt).
3. Computes a **grounding score** — the fraction of factual claims that can be attributed to retrieved sources.
4. Produces a `ReasonedAnswer` with `SourceCitation` objects linking each claim to its evidence.

Why grounding matters: Ungrounded RAG is just a chatbot with a bigger context window. Grounding transforms RAG from "the model said it" to "the evidence supports it." In legal and regulatory contexts, this is the difference between a useful tool and a liability.

6.4 Temporal Consistency & Claim Validation

Two post-inference validators act as quality gates:

- **TemporalConsistencyValidator:** Checks that the answer doesn't make temporally impossible claims (e.g., referencing a document version that didn't exist at a given date, or claiming an event occurred before its documented start).
- **ClaimValidator:** Cross-references factual claims against authority references from the knowledge graph. If the LLM claims "Regulation X requires Y" but the knowledge graph shows Regulation X says Z, the claim is flagged.

When either validator detects an issue, a warning is appended to the answer rather than blocking it — this is a deliberate UX choice that gives users transparency into the AI's limitations.

6.5 Confidence Scoring

Every AI answer carries a `ConfidenceProfile` with:

- **Overall confidence:** 0.0–1.0 score aggregating grounding, temporal, and claim validation.
- **Source coverage:** What fraction of the retrieved context was actually used.
- **Citation density:** Number of distinct sources cited per factual claim.

7. Knowledge Graph (Neo4j)

7.1 Role of the Knowledge Graph

The Neo4j knowledge graph serves a **distinct purpose from document search**. While the search system retrieves document passages relevant to a query, the knowledge graph models **structured relationships between authoritative knowledge entries** — statutes, regulations, reference frameworks, taxonomies.

It enables queries like:

- "Show me all statutes related to this one via shared legal concepts."
- "Traverse the regulatory hierarchy from broad provision to specific clause."
- "What other entries share concepts with Entry 123?"

7.2 Graph Data Model

Nodes:

```
(:KnowledgeEntry {
  canonicalReference: "CA Civ Code § 1542",
  domainCode: "CA_CIV",
  domain: "California Civil Code",
  entryNumber: "1542",
  entryTitle: "General Release - Claim Not Known to Exist",
  hierarchyPath: "CA_CIV > Division 3 > Part 1 > Title 10 > § 1542",
  indexedAt: datetime()
})
```

Relationships:

```
(child)-[:PART_OF_HIERARCHY {level: "section"}]→(parent)
(entry1)-[:RELATED_CONCEPT {concept: "waiver"}]→(entry2)
(entry1)-[:RELATED_CONCEPT {concept: "release_of_claims"}]→(entry3)
```

7.3 Integration Pattern

The `KnowledgeGraphService` wraps the Neo4j Java Driver and provides methods for:

- `createEntryNode()` — Upsert a knowledge entry as a graph node.
- `createHierarchyRelationship()` — Link entries in parent-child hierarchy.
- `createConceptRelationship()` — Link entries sharing a concept.
- `traverseByConcepts()` — Graph traversal with configurable depth.

The service is **resilience-aware**: it verifies Neo4j connectivity at startup and gracefully degrades if Neo4j is unavailable (`isAvailable()` returns false). All graph operations are no-ops when the database is unreachable, ensuring the platform remains functional without the knowledge graph.

8. Data & Persistence Layer

8.1 PostgreSQL Schema Strategy

The platform uses **Hibernate's** `ddl-auto: update` for development agility, with the understanding that a production deployment would use Flyway migrations (already configured as a dependency) for version-controlled schema evolution.

Key entities stored in PostgreSQL:

- `UserAccountEntity` — User credentials, roles, account status
- `RefreshTokenSessionEntity` — Active JWT refresh tokens
- `DocumentEntity` — Document metadata, status, type, timestamps
- `DocumentVersionEntity` — Version history per document
- `DocumentChunkEntity` — Text chunks with positions, metadata, keywords
- `IngestionJobEntity` — Async ingestion job tracking
- `WorkspaceEntity` — Workspace metadata and status
- `WorkspaceDocumentLinkEntity` — Many-to-many workspace-document links
- `WorkspaceStepEntity` — Workspace phase tracking
- `TimelineEventEntity` — Extracted timeline events from documents
- `KnowledgeEntryEntity` — Structured knowledge entries
- `AuditEvent` (via JDBC) — Immutable audit trail

8.2 Why We Need a Relational Database

Vector databases and knowledge graphs are specialized tools — they don't replace a relational database for structured data. PostgreSQL provides:

- **Referential integrity:** A document version cannot exist without its parent document. A workspace-document link must reference existing entities. Foreign keys enforce this.
- **Transactional consistency:** Uploading a document involves creating a Document row, a DocumentVersion row, N DocumentChunk rows, and an IngestionJob row. If any step fails, the entire transaction rolls back.
- **Ad-hoc querying:** "Show me all documents uploaded in the last 30 days by users in the Legal group" — this is a 10-second JPA Specification, not something you'd model in a vector database or graph.
- **Aggregation & reporting:** COUNT, SUM, GROUP BY — these are SQL primitives. Neo4j and Qdrant are not designed for aggregate reporting.

8.3 Document Storage & Versioning

Documents are stored on the filesystem (configurable path) with database metadata. The versioning model:

- Every document has an integer `version` counter.
- Adding a new version increments the counter and stores the new file alongside previous versions.
- Each version has its own chunks and embeddings, enabling version-specific search.
- Old chunks are cleaned up from Qdrant when a document is deleted or replaced.

9. Security Architecture

9.1 JWT-Based Authentication

The platform implements a dual authentication mechanism:

- **Form login:** Traditional Thymeleaf form POST to `/login` with Spring Security's form login filter. Used by the browser UI.
- **JWT resource server:** REST API endpoints accept Bearer tokens issued by the platform's own JWT encoder. Enables programmatic API access.

Token architecture:

- **Access tokens:** Short-lived (15 minutes), HMAC-SHA256 signed, contain subject, roles, and issuer claims.
- **Refresh tokens:** Long-lived (30 days), stored in database with session metadata, can be revoked server-side.
- **Token rotation:** Each refresh issues a new access token and refresh token, invalidating the previous refresh token.

9.2 Authorization Model

```
PUBLIC ROUTES:      /login, /register, /css/**, /actuator/health
AUTHENTICATED:     /dashboard, /workspaces, /documents, /search, /ai, /audit
ADMIN ONLY:        /api/admin/**
REST API:           /api/auth/** (JWT Bearer token)
```

Role-based access control via the `roles` JWT claim, mapped to Spring Security `ROLE_` granted authorities.

Method-level security via `@EnableMethodSecurity` allows fine-grained authorization on service methods.

9.3 Audit Trail System

The audit module (`platform-audit`) provides comprehensive event logging:

- **Event types:** `AUTH_LOGIN`, `AUTH_REGISTER`, `DOCUMENT_UPLOADED`, `DOCUMENT_VERSION_ADDED`, `INGESTION_STARTED`, `INGESTION_COMPLETED`, `SEARCH_EXECUTED`, `RETRIEVAL_EXECUTED`, `MODEL_INFERENCE`, `PROMPT_EXECUTED`, `WORKSPACE_CREATED`, and many more.
- **Correlation IDs:** Every request gets a unique correlation ID (via `CorrelationIdFilter`) that flows through all services and appears in audit events — enabling end-to-end traceability.

- **Structured metadata:** Each event carries a `Map<String,String>` of typed metadata (e.g., model name, prompt version, retrieval source count).
- **Immutable storage:** Audit events are append-only via JDBC. No updates, no deletes.

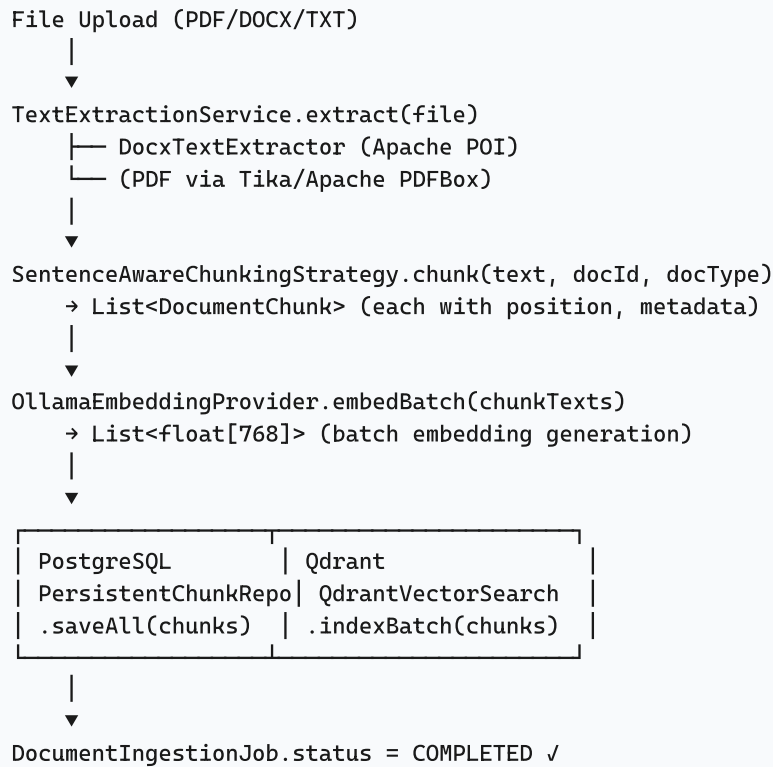
10. Internationalization (i18n)

The platform supports **three languages** (English, German, French) using Spring's `MessageSource` and Thymeleaf's `#key` syntax:

- **Message files:** `messages.properties`, `messages_de.properties`, `messages_fr.properties` with 100+ translated keys covering navigation, forms, error messages, and AI interaction labels.
- **Session-based locale:** `SessionLocaleResolver` stores the user's language preference. The `LocaleChangeInterceptor` switches locale via the `?lang=` query parameter.
- **Language switcher:** A dropdown in the navigation bar renders flag icons with links to the current page in each language.
- **Dual-module i18n:** Both `platform-api` and `platform-web` maintain their own message bundles, allowing independent deployment of the REST API and web frontend.

11. Ingestion Pipeline

Document ingestion is the process that transforms uploaded files into searchable, AI-ready chunks:



Key design decisions in the ingestion pipeline:

- **Async processing:** Ingestion runs in a background worker (`DocumentIngestionWorker`) so file upload returns immediately. Users monitor progress via the Jobs page.
- **Batch embedding:** Embeddings are generated in batches for efficiency, not one chunk at a time.
- **Dual indexing:** Every chunk is indexed in both PostgreSQL (for keyword search + persistence) and Qdrant (for vector search). This ensures both search paths have complete coverage.

12. Workspace System

Workspaces provide **organizational context** for document analysis. A workspace groups related documents and guides users through a structured workflow:

Phase	Description	Actions
SETUP	Define workspace purpose, type, and initial configuration	Create workspace, set metadata
UPLOAD	Add documents to the workspace	Upload files, attach existing documents
INGEST	Process and index documents	Trigger ingestion, monitor jobs
ANALYSIS	AI-powered analysis of workspace documents	Run AI queries scoped to workspace
REVIEW	Review findings, validate results	Browse AI answers, check citations
COMPLETE	Workspace archived, results finalized	Export results, mark complete

The `WorkspaceOrchestrator` manages phase transitions through a pluggable handler system — each phase has a registered `PhaseHandler` implementation, making it easy to add custom phases for specific use cases.

13. Testing Strategy

The platform maintains **88 passing tests** across multiple layers:

Test Type	Examples	Coverage
Integration Tests	PlatformIntegrationTest , AuthIntegrationTest	Full Spring context with test database
Workflow Tests	UploadWorkflowTest , WorkspaceWorkflowTest	End-to-end user workflows
Navigation Tests	NavigationMenuTest , PageAccessTest	Page rendering, access control, menu structure
I18n Tests	I18nIntegrationTest	Multi-language message resolution
UI Layout Tests	DashboardLayoutTest	Thymeleaf template rendering validation
Unit Tests	(via Spring Boot Test slices)	Service logic isolation

14. Deployment & Operations

The platform is designed for **Docker-based deployment** with four infrastructure services:

```
# Bring up all infrastructure
docker compose up -d

# Services:
# PostgreSQL → localhost:5433 (pgvector/pgvector:pg16)
# Qdrant      → localhost:6333 (REST), localhost:6334 (gRPC)
# Neo4j       → localhost:7474 (Browser), localhost:7687 (Bolt)
# PGAdmin     → localhost:5050 (optional management UI)

# Start Ollama separately (GPU-accelerated)
ollama serve
ollama pull qwen2.5:14b
ollama pull nomic-embed-text

# Run the application
./mvnw -pl platform-api spring-boot:run
```

Configuration is externalized via environment variables:

```
DB_URL, DB_USERNAME, DB_PASSWORD
NEO4J_URI, NEO4J_USER, NEO4J_PASSWORD
QDRANT_HOST, QDRANT_REST_PORT, QDRANT_COLLECTION
OLLAMA_BASE_URL, OLLAMA_EMBEDDING_MODEL, OLLAMA_CHAT_MODEL
AUTH_JWT_SECRET, AUTH_ACCESS_TOKEN_TTL, AUTH_REFRESH_TOKEN_TTL
```

This design supports deployment to any environment — local development, on-premises servers, or Kubernetes — by changing environment variables without modifying application code.

15. Why This Architecture Excels

This architecture embodies several principles that distinguish production AI systems from prototypes:

1. Data Sovereignty by Default

Every component — the LLM (Ollama), vector database (Qdrant), graph database (Neo4j), and relational database (PostgreSQL) — runs on the organization's own infrastructure. No data leaves the premises. This is not a feature; it's an architectural constraint that drove every technology choice.

2. Graceful Degradation

Every external service dependency is optional. If Neo4j is down, graph features are disabled but the platform works. If Qdrant is down, keyword search still works. If Ollama is unavailable, the platform clearly reports the issue rather than crashing. This resilience is built into each service adapter.

3. Provider Abstraction

The `ChatCompletionProvider`, `EmbeddingProvider`, `VectorSearchProvider`, and `KeywordSearchProvider` interfaces mean the platform can switch LLM providers, embedding models, or vector databases without changing business logic. This is the **Dependency Inversion Principle** applied to AI infrastructure.

4. Auditability as a First-Class Concern

Every AI answer records the prompt template version, retrieval sources, model used, and inference metadata. Every search records the mode, intent, and result counts. This is not an afterthought — it's built into the service interfaces and published through the audit system.

5. Intent-Driven Query Routing

The dual `QueryIntentClassifier` (one in platform-search, one in platform-ai) ensures queries are handled by the most appropriate subsystem. Simple index queries don't waste LLM tokens. Complex analytical queries get the full RAG pipeline. This is cost-aware architecture.

6. Correctness Over Velocity

Java + Spring Boot + static typing + ACID transactions were chosen over faster-to-prototype alternatives because the domain (legal/compliance document analysis) demands correctness. An AI that hallucinates a statute citation is worse than no AI at all. The grounding, validation, and audit systems are safeguards against this failure mode.

16. Key Design Decisions Summary

Decision	Chosen	Rejected Alternatives	Primary Reason
Application Framework	SPRING BOOT 3.3	FastAPI, Express, Django	Type safety, transaction management, enterprise ecosystem
LLM Runtime	OLLAMA (LOCAL)	OpenAI API, Anthropic API, vLLM	Data sovereignty, zero API cost, model flexibility
UI Framework	THYMELEAF + BOOTSTRAP	React, Angular, Vue, HTMX	SSR, no JS build step, Spring Security integration, i18n
Vector Database	QDRANT	Pinecone, Weaviate, Milvus, Chroma	Self-hosted, Rust performance, clean REST API
Graph Database	NEO4J 5	PostgreSQL CTEs, Amazon Neptune, ArangoDB	Native graph traversal, Cypher expressiveness, APOC plugins
Relational Database	POSTGRESQL + PGVECTOR	MySQL, MongoDB, CockroachDB	ACID, pgvector extension, Spring Data JPA maturity
Search Strategy	HYBRID (KEYWORD + VECTOR)	Keyword-only, vector-only	Best of both: precision + recall, with intent-aware fusion
AI Pipeline	RAG + GROUNDING + VALIDATION	Pure LLM, fine-tuned model, agent loop	Evidence-backed, auditable, hallucination-resistant
Language	JAVA 21 LTS	Python, TypeScript, Go, Kotlin	Static typing, virtual threads, enterprise library ecosystem
Architecture Style	MODULAR MONOLITH	Microservices, serverless	Deployment simplicity, transactional integrity, low operational overhead

Parting thought for the AI consultant interview: This platform demonstrates that production AI is not about picking the trendiest framework or the biggest model. It's about making deliberate, justified architectural choices that prioritize correctness, data sovereignty, auditability, and resilience. Every component has a clear reason for being there. Every abstraction maps to a real requirement. That's the difference between a prototype and a platform.